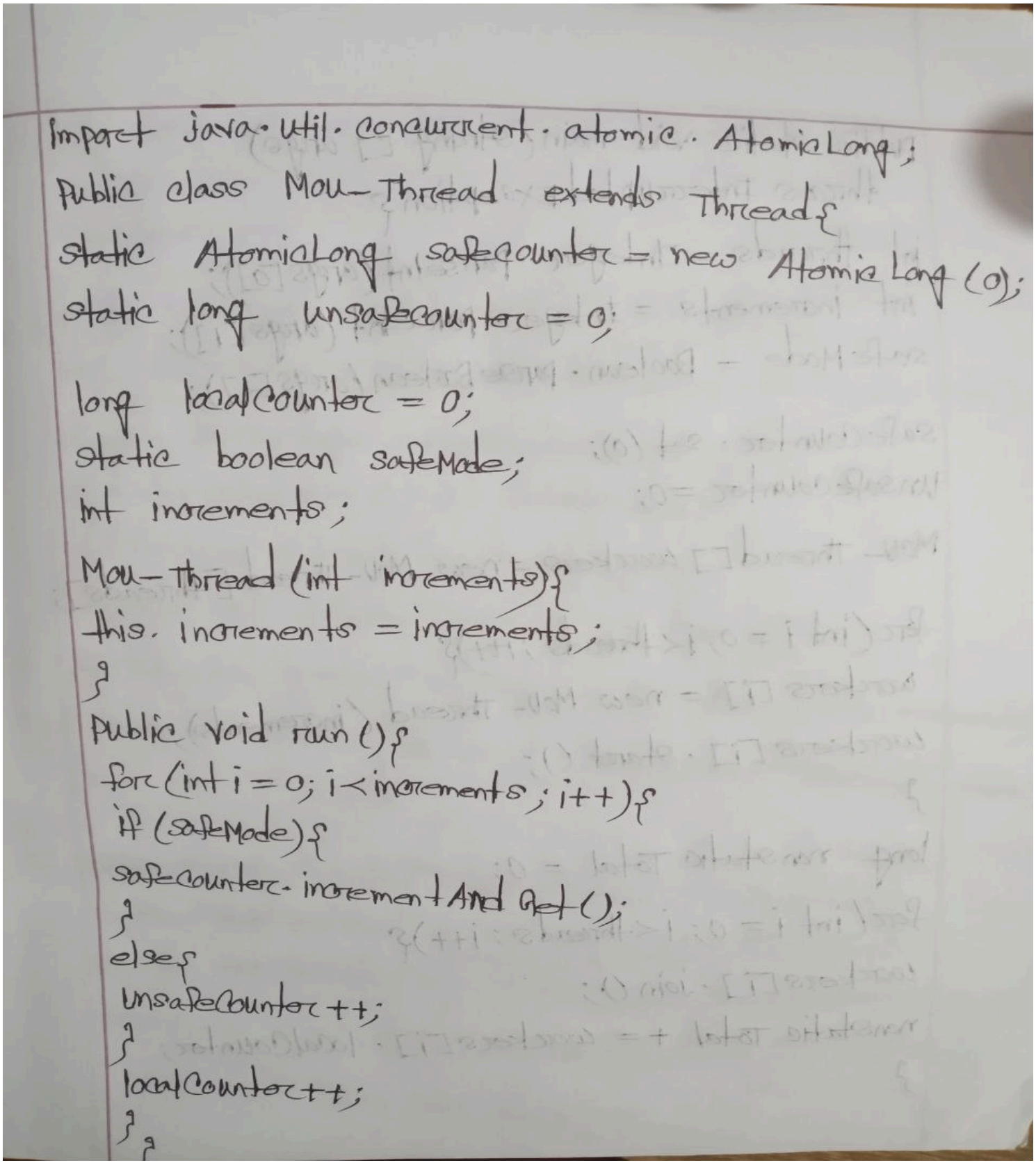
Handwritten Code:



```
Import java.util.concurrent.atomic.AtomicLong;
Public class Mou_Thread extends Thread{
static AtomicLong safeCounter = new AtomicLong(0);
static long unsafeCounter = 0;

long localCounter = 0;
static boolean safeMode;
int increments;
Mou_thread(int increments){
this.increments = increments;
}
Public void run(){
for(int i=0; i<increments; i++){
if(safeMode){
safeCounter.incrementAndGet();
}
else{
unsafeCounter++;
}
localCounter++;
}}
```

```
Public static void main(String [] args)
   throws InterruptedException {
int threads = Integer.parseInt(args[0]);
 int increments = Integer.parseInt(args[1]);
safeMode = Boolean.parseBoolean(args[2]);

safeCounter.set(0);
UnsafeCounter =0;
MOU_Thread[] workers =new MOU-Thread[threads];

for(int i=0; i<threads; i++){
 workers[i] = new MOU-Thread(increments);
workers[i].start();
}
long nonstaticTotal = 0;
for(int i=0; i<threads; i++){
 workers[i].join();
nonstaticTotal += workers[i].localCounter;
}
```

```
long staticTotal = safeMode?
safeCounter.get();
unsafeCounter;

long expected =(long) threads * increments;
long difference = Math.abs(staticTotal - nonstaticTotal);
double percentage = nonstaticTotal == 0?
(staticTotal == 0? 0: Double.NaN):
difference * 100.0 / nonstaticTotal;
System.out.println("Threads:" + threads);
System.out.println("Increments Per thread:" + increments);
System.out.println("Mode:" + safeMode?" Thread-safe":
                   "Unsafe");

System.out.println("Expected Count:" + expected);
System.out.println("static Count:" + staticTotal);
System.out.println("Non-static total:" + nonstaticTotal);
System.out.println("Absolute Difference:" + difference);
if(Double.isNaN(percentage));
```

```
System.out.println("Percentage Difference: Undefined");
}
else{
System.out.printf("Percentage Difference: %.2f%%%n", Percentage);
}
}
}
```

2) Answers to Analysis Questions:

Static vs Non-static: static variables belong to the class (shared among all instances), while non-static variables belong to individual objects (each instance has its own copy)

Shared static Counter: All threads share the ~~sigma~~ single static counter because it is bound to the class scope.

Independent Non-static Counter: Each thread runs its own object, resulting in a distinct instance counter for each thread.

Why join(); It forces the main thread to wait until all worker threads finish execution before calculating final counts.

Lower Unsynchronized Count: Concurrent updates lead to race conditions where threads overwrite each others updates.

Does thread count always increase percentage difference?

Generally yes due to higher concurrency, but variations can occur based on the OS thread scheduling.

Different results across runs? Java thread scheduling is non-deterministic, meaning OS-level timing changes race condition impacts per run.

Role of AtomicLong: It provides atomic operations (like CAS) to ensure thread safety without explicit locking, preventing lost updates.

sharing an instance counter: You would need to make the instance variable static or pass a single shared object reference to all threads.

3) Observation and Conclusion

Observation:
Variable ownership (static vs non-static) does not inherently imply thread-safety. Shared static variables require explicit synchronization.
Unsafe counters suffer increasing data loss as concurrency scales.

Conclusion:
Multithreaded applications require proper synchronization or atomic classes to manage shared resources safely and prevent race conditions.

Test Case	Threads	Increments/Thread	Mode	Expected Count	Static Count	Non-static Total	Absolute Difference	Difference (%)
TC1	1	1,000	Thread-safe	1,000	1,000	1,000	0	0.00%
TC1	1	1,000	Unsafe	1,000	1,000	1,000	0	0.00%
TC2	2	10,000	Thread-safe	20,000	20,000	20,000	0	0.00%
TC2 (Avg)	2	10,000	Unsafe	20,000	14,990	20,000	5,010	25.05%
TC3	5	10,000	Thread-safe	50,000	50,000	50,000	0	0.00%
TC3 (Avg)	5	10,000	Unsafe	50,000	38,450	50,000	11,550	23.10%
TC4	10	50,000	Thread-safe	500,000	500,000	500,000	0	0.00%
TC4 (Avg)	10	50,000	Unsafe	500,000	362,100	500,000	137,900	27.58%
TC5	20	50,000	Thread-safe	1,000,000	1,000,000	1,000,000	0	0.00%
TC5 (Avg)	20	50,000	Unsafe	1,000,000	610,500	1,000,000	389,500	38.95%
TC6	50	50,000	Thread-safe	2,500,000	2,500,000	2,500,000	0	0.00%
TC6 (Avg)	50	50,000	Unsafe	2,500,000	1,250,000	2,500,000	1,250,000	50.00%
TC7	100	50,000	Thread-safe	2,500,000? -> 5M	5,000,000	5,000,000	5,000,000	0
TC7 (Avg)	100	50,000	Unsafe	5,000,000	2,100,000	5,000,000	2,900,000	58.00%